



United International University (UIU)
Department of Computer Science and Engineering (CSE)
Course: VLSI Design (CSE 4327)

Experiment 6: Synthesis of a 4-bit Counter using RTL Compiler

Objectives

1. To understand the standard-cell synthesis flow: translating an RTL Verilog description into a gate-level netlist using a target technology library.
2. To simulate and verify a 4-bit counter design before proceeding to synthesis.
3. To write timing constraints (SDC) for a design and use them to guide synthesis.
4. To use Cadence RTL Compiler (rc) to synthesize the 4-bit counter, and to examine its timing and power reports.
5. To view the synthesized gate-level schematic and to generate a gate-level netlist and SDC file suitable for physical design.

Introduction

Standard-cell synthesis is the process of translating a technology-independent RTL (Verilog) description of a circuit into a gate-level netlist built from cells in a target standard-cell library, guided by timing constraints such as clock period and input/output delays. Cadence RTL Compiler (invoked as rc) is the synthesis tool used for this purpose, playing a role in the digital design flow analogous to Schematic XL and ADE L for schematic capture and simulation, and Layout XL and PVS for physical layout and verification, covered in earlier experiments.

In this experiment, a 4-bit up-counter - a synchronous sequential circuit similar to the one designed in Experiment 4 - is carried through a complete standard-cell design flow: the RTL is first simulated and verified in isolation, then synthesized against a technology library and a set of timing constraints, and finally examined as a mapped gate-level schematic, netlist, and SDC file ready for physical design.

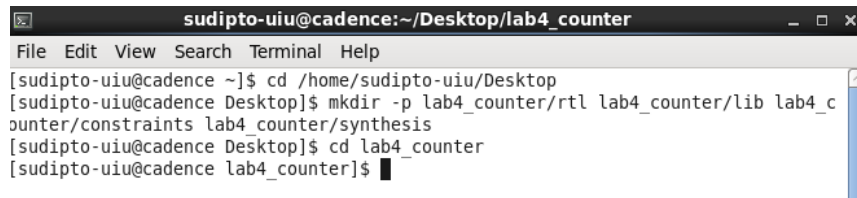
Note: This experiment assumes your Cadence tools environment (cshrc/csh) has already been configured, as covered in the earlier labs. If you are starting a fresh session, remember to switch to csh before invoking any Cadence tool.

Note: The reference screenshots in this manual were captured on a machine where the working folder was named lab4_counter, and paths are shown under that user's own home directory (/home/sudipto-uiu/Desktop). Use lab6 as your own working folder name, as instructed below, and substitute your own username and home directory throughout; the directory structure and commands are otherwise identical.

Part 1: Setting Up the Design Directory

1. Create the lab directory and its sub-directories under your work area. In addition to rtl, lib, constraints, and synthesis, create a simulation sub-directory, since the design will be simulated and verified before it is synthesized:

```
cd ~/Desktop
mkdir -p lab6/rtl lab6/lib lab6/constraints lab6/synthesis lab6/simulation
cd lab6
```

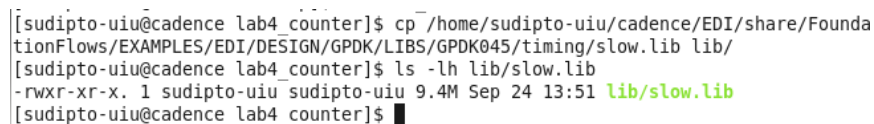


```
sudipto-uiu@cadence:~/Desktop/lab4_counter
File Edit View Search Terminal Help
[sudipto-uiu@cadence ~]$ cd /home/sudipto-uiu/Desktop
[sudipto-uiu@cadence Desktop]$ mkdir -p lab4_counter/rtl lab4_counter/lib lab4_c
ounter/constraints lab4_counter/synthesis
[sudipto-uiu@cadence Desktop]$ cd lab4_counter
[sudipto-uiu@cadence lab4_counter]$
```

Figure 1: Terminal session creating the working folder and its sub-directories (shown here as lab4_counter; use lab6 for your own work).

2. Copy the slow timing library into your lib/ sub-directory, and confirm it copied correctly:

```
cp /home/sudipto-
uii/cadence/EDI/share/FoundationFlows/EXAMPLES/EDI/DESIGN/GPDK/LIBS/GPDK045/timing
/slow.lib lib/
ls -lh lib/slow.lib
```



```
[sudipto-uiu@cadence lab4_counter]$ cp /home/sudipto-uiu/cadence/EDI/share/Founda
tionFlows/EXAMPLES/EDI/DESIGN/GPDK/LIBS/GPDK045/timing/slow.lib lib/
[sudipto-uiu@cadence lab4_counter]$ ls -lh lib/slow.lib
-rwxr-xr-x. 1 sudipto-uiu sudipto-uiu 9.4M Sep 24 13:51 lib/slow.lib
[sudipto-uiu@cadence lab4_counter]$
```

Figure 2: Copying slow.lib into lib/ and confirming the file with ls -lh.

Note: The exact path to the GPDK045 timing library may differ depending on your installation. If the copy command fails, ask the lab instructor for the correct FoundationFlows path on your machine.

Part 2: Writing and Simulating the 4-bit Counter

Before any design is synthesized, it must first be simulated and verified in isolation, exactly as in the earlier experiments. Only once you are satisfied that the RTL behaves correctly should it be carried forward into the synthesis flow.

3. Use an editor (vi, nedit, gedit, etc.) to write the RTL model of a 4-bit counter. Save it in your simulation directory as cntr4bit.v:

```
gedit simulation/cntr4bit.v &
```

```
`timescale 1ns/1ps

module cntr4bit (
    input wire clk,
    input wire rst,
    output reg [3:0] count
);

    always @(posedge clk or negedge rst) begin
        if (!rst)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
        end
endmodule
```

Listing 1: cntr4bit.v - 4-bit synchronous up-counter with active-low asynchronous reset. It counts 0, 1, 2, ..., 15, 0, ...; rst = 0 resets it, as in the earlier counter experiment.

4. Use an editor to write a test bench Verilog code for your counter. Save it in your simulation directory as cntr4bit_tb.v:

gedit simulation/cntr4bit_tb.v &

```
`timescale 1ns/1ps

module cntr4bit_tb;

    reg clk;
    reg rst;
    wire [3:0] count;
    integer i;

    // Connect the testbench to the counter
    cntr4bit dut (
        .clk(clk),
        .rst(rst),
        .count(count)
    );

    // Clock period = 10 ns
    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst = 0;                // Apply active-low reset

        #2;
        if (count != 4'd0) begin
            $display("FAIL: Reset did not clear the counter");
            $finish;
        end

        $display("Reset OK: count = %d", count);
        rst = 1;                // Release reset

        // Check 17 clock edges: includes 15 -> 0 rollover
        for (i = 1; i <= 17; i = i + 1) begin
            @(posedge clk);
            #1;                // Let the counter update after the clock edge

            if (count != (i % 16)) begin
                $display("FAIL at cycle %0d: expected %0d, got %0d",
                    i, i % 16, count);
                $finish;
            end

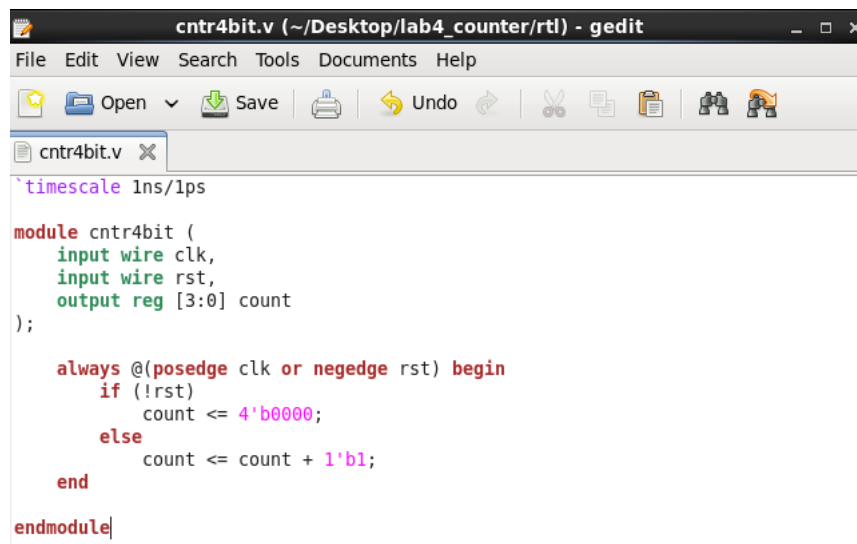
            $display("Cycle %0d: count = %0d", i, count);
        end

        $display("PASS: reset, counting, and rollover all work");
        $finish;
    end

endmodule
```

Listing 2: cntr4bit_tb.v - self-checking test bench that verifies the reset value, counts through 17 clock edges (to include the 15 $\rightarrow$ 0 rollover), and reports PASS or FAIL.

5. Use the Incisive Enterprise Simulator (IES) to verify that the counter is working properly. Follow the compile - elaborate - simulate steps described in Experiment 4 (NCLaunch $\rightarrow$ create the work library $\rightarrow$ ncvlog $\rightarrow$ ncelab $\rightarrow$ launch simulator), working from your simulation directory. Confirm in the SimVision waveform, or in the \$display output in the Command Entry Window, that the test bench reports PASS.
6. When you are satisfied with your simulation, copy the counter module into the rtl sub-directory:
cp simulation/cntr4bit.v rtl/



```

cntr4bit.v (~/Desktop/lab4_counter/rtl) - gedit
File Edit View Search Tools Documents Help
[Icons: Open, Save, Undo, Redo, Cut, Copy, Paste, Find, Run, etc.]
cntr4bit.v x
`timescale 1ns/1ps

module cntr4bit (
    input wire clk,
    input wire rst,
    output reg [3:0] count
);

    always @(posedge clk or negedge rst) begin
        if (!rst)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
    end

endmodule

```

Figure 3: cntr4bit.v opened in gedit from the rtl/ directory, after the verified design has been copied there.

Part 3: Creating the Timing Constraints

7. Create the design constraint (SDC) file that will guide synthesis. Open the file:

gedit constraints/cntr4bit.g &

Paste the following contents and save it:

```

create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.1 [get_clocks "clk"]
set_input_delay -max 1.0 [get_ports "rst"] -clock [get_clocks "clk"]
set_output_delay -max 1.0 [get_ports "count"] -clock [get_clocks "clk"]

```

Listing 3: cntr4bit.g - a 10 ns clock definition, clock transition and uncertainty settings, and input/output delay constraints for rst and count.

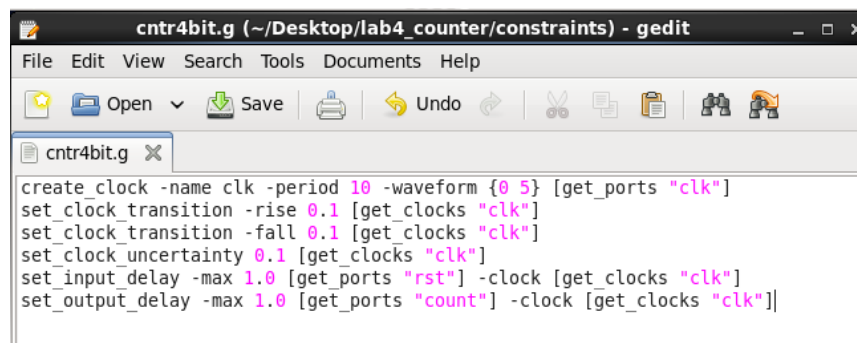


Figure 4: cntr4bit.g open in gedit, showing the clock definition and I/O delay constraints.

Part 4: Creating the Synthesis Script

8. Create the RTL Compiler synthesis script. Open the file:

gedit synthesis/rc_script.tcl &

Paste the following contents and save it:

```

set_attr lib_search_path ../lib/
set_attr hdl_search_path ../rtl/
set_attr library slow.lib

read_hdl -v2001 cntr4bit.v
elaborate cntr4bit
read_sdc ../constraints/cntr4bit.g

synthesize -to_mapped -effort medium

report timing
report power

write_hdl > cntr4bit_netlist.v
write_sdc > cntr4bit.sdc

```

Listing 4: rc_script.tcl - sets the library and search paths, reads and elaborates the RTL, reads the SDC constraints, synthesizes to a mapped netlist, reports timing and power, and writes out the netlist and SDC.

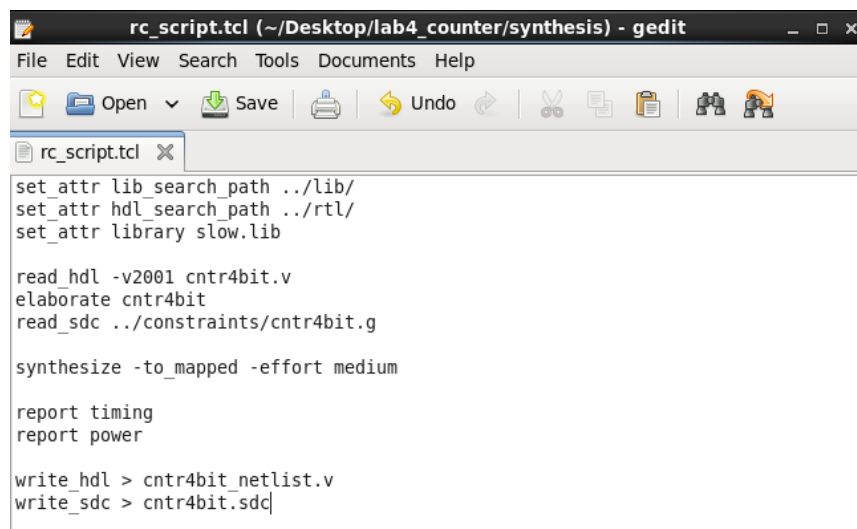


Figure 5: rc_script.tcl open in gedit, showing the complete synthesis script.

Note: The -v2001 option on read_hdl is necessary for this design: it selects Verilog-2001 port declaration syntax (the ANSI-style input wire clk, ... port list used in cntr4bit.v) and avoids the port declaration error that can otherwise occur.

Part 5: Checking the Files Before Running Synthesis

9. From your lab6 directory, confirm that all four required files are in place:

```
ls -lh rtl/cntr4bit.v lib/slow.lib constraints/cntr4bit.g synthesis/rc_script.tcl
```

```
[sudipto-uiu@cadence lab4_counter]$ ls -lh rtl/cntr4bit.v lib/slow.lib constraint
s/cntr4bit.g synthesis/rc_script.tcl
-rw-rw-r--. 1 sudipto-uiu sudipto-uiu 354 Sep 24 13:52 constraints/cntr4bit.g
-rwxr-xr-x. 1 sudipto-uiu sudipto-uiu 9.4M Sep 24 13:51 lib/slow.lib
-rw-rw-r--. 1 sudipto-uiu sudipto-uiu 271 Sep 24 13:52 rtl/cntr4bit.v
-rw-rw-r--. 1 sudipto-uiu sudipto-uiu 297 Sep 24 13:53 synthesis/rc_script.tcl
[3]+ Done gedit synthesis/rc_script.tcl
[sudipto-uiu@cadence lab4_counter]$
```

Figure 6: `ls -lh` confirming that `cntr4bit.v`, `slow.lib`, `cntr4bit.g`, and `rc_script.tcl` are all present.

Part 6: Running Synthesis

10. Move into the synthesis directory and invoke RTL Compiler with the synthesis script:

```
cd synthesis
rc -f rc_script.tcl
```

11. While synthesis is running, watch the rc terminal for any Error messages. A Warning about possible timing problems is expected here and does not indicate a failure; read it to understand what it is reporting.

```

sudipto-uiu@cadence:~/Desktop/lab4_c
File Edit View Search Terminal Help

init_drc      0      0      0      0
init_area    0      0      0      0

Done mapping cntr4bit
Synthesis succeeded.
Warning : Possible timing problems have been detected in this design. [TIM-11]
: The design is 'cntr4bit'.
: Use 'report timing -lint' for more information.

-----
Generated by:      Encounter(R) RTL Compiler v12.10-s012.1
Generated on:      Sep 24 2020 01:54:37 pm
Module:           cntr4bit
Technology library: gpdK045bc
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
-----

Pin              Type      Fanout Load Slew Delay Arrival
              (fF) (ps)  (ps)  (ps)
-----
{clock clk}      launch              0 R
count_reg[3]/CK  100              0 R
count_reg[3]/Q   SDFFRX4      2 0.0 42 +330 330 R
count[3]         out port              +0 330 R
{cntr4bit.g_line_0} ext delay              +1000 1330 R
-----
{clock clk}      capture              10000 R
uncertainty              -100 9900 R
-----

Cost Group : 'clk' (path_group 'clk')
Timing slack : 8570ps
Start-point : count_reg[3]/CK
End-point : count[3]

-----
Generated by:      Encounter(R) RTL Compiler v12.10-s012.1
Generated on:      Sep 24 2020 01:54:37 pm
Module:           cntr4bit
Technology library: gpdK045bc
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
-----

Leakage Dynamic Total
Instance Cells Power(nW) Power(nW) Power(nW)
-----
cntr4bit 7 3.021 8307.397 8370.418

Finished SDC export (command execution time mm:ss (real) = 00:00).
rc:/>

```

Figure 7: rc terminal output at the end of synthesis, showing the timing report, the power report, and confirmation that the SDC export finished.

12. When the script completes, confirm that the netlist and SDC files were generated:

```
ls -lh cntr4bit_netlist.v cntr4bit.sdc
```

Enter this ls command at the Linux prompt (after leaving RTL Compiler with exit), not at the rc prompt.

13. To see the synthesized cells as text, view the netlist:

```
cat cntr4bit_netlist.v
```

Expect to see four flip-flops, plus logic cells that compute the next count value. The exact cells chosen (their drive strength and gate type) are Cadence's decision, based on the technology library and constraints.

Part 7: Viewing the Schematic in the GUI

14. After synthesis, launch the GUI to see the schematic. If you are still at the rc prompt:

```
gui_show
```

If you have already returned to the Linux prompt, start an interactive RTL Compiler session from the synthesis folder instead, then source the script and open the GUI:

```
rc
```

```
source rc_script.tcl
```

```
gui_show
```

15. In the GUI, locate `cntr4bit` in the design browser and open its Schematic view. Zoom in to identify the four flip-flops and the logic feeding their D inputs.

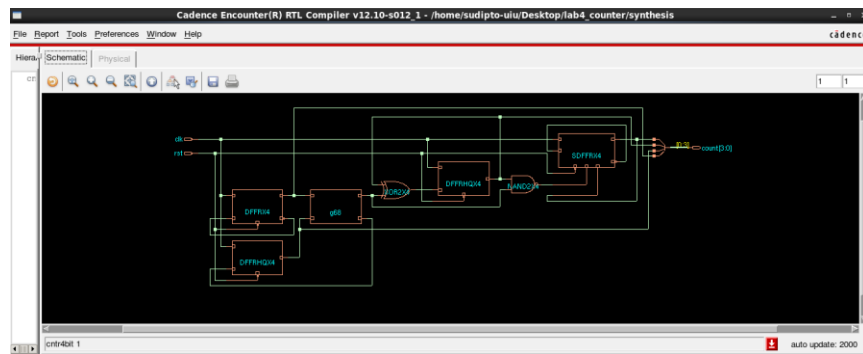


Figure 8: Cadence RTL Compiler schematic view of the synthesized `cntr4bit` design, showing the four flip-flops (DFFRX4 / DFFRHQX4 / SDFFRX4) and the gates that compute each next-state D input.

16. After checking the schematic, if you are satisfied, close the GUI and leave the tool from the rc prompt:

```
gui_hide
exit
```

17. Use the report command to write out further results at any point while still inside RTL Compiler:

```
report timing -- reports the timing details
report power  -- writes the power report
report        -- lists the different reports you can dump from RC
```

Both `report timing` and `report power` are already part of `rc_script.tcl`, so they run automatically each time you execute the script; the report command above is how you can re-run them, or explore other reports, interactively.

18. Open the Cadence Help GUI to explore RTL Compiler further:

```
cdnshelp &
```

Expand RTL Compiler in the tree and double-click on any sub-menu you would like to explore.

Credits

This lab manual was prepared and formatted by:

Sudipto Mondal

Lecturer, Department of Computer Science and Engineering (CSE)

United International University (UIU)

Email: sudiptomondal@cse.uiu.ac.bd